

21/1/26

Types of Number Systems

- 1, Decimal (10) - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
 - 2, Octal (8) - 0, 1, 2, 3, 4, 5, 6, 7
 - 3, Binary (2) - 0, 1
 - 4, Hexadecimal (16) - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- The no. of distinct numbers are used, that is called Base (or) Radix

Nibble - 4 bits
Bytes - 8 bits

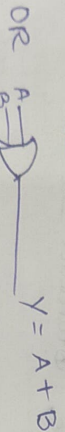
→ Gate is a hardware circuit which give output 0 or 1 based on its input is called Gate

Basic Gates

Three Basic Gates



0 is low
1 is high



→ Suppose there are 2 inputs, there will be 2ⁿ combinations

AND

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

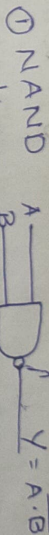
OR

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

NOT

A	Y
0	1
1	0

UNIVERSAL GATES



AND + NOT



OR + NOT

→ By using NAND or NOR gates we can realize all the basic gates so they are called as Universal gates.

→ The output is high when all of its inputs are high

called AND Gate (similar definitions for all gates)

→ The output is high when one of its input is high called OR Gate

Exclusive OR Gate

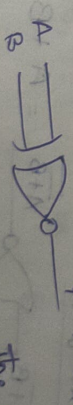


$$A \oplus B = \bar{A}B + A\bar{B}$$

If all the inputs are same then the output is low

Capital letter A, B, C, D
Small letter a, b, c, d

Ex- NOR Gate



The output is low when all of its inputs are high called NAND

① $\overline{A+B} = \bar{A} \cdot \bar{B}$

② $\overline{A \cdot B} = \bar{A} + \bar{B}$

The output is low when all of its inputs are low called NOR

23/1/26 Prove that NAND as Universal gates.

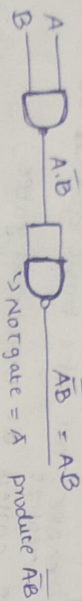
NAND

① NAND as a NOT gate

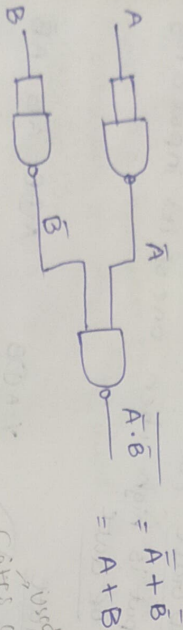


$$A=B \Rightarrow A$$

② NAND as a AND gate



③ NAND as a OR gate



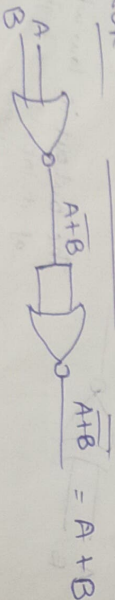
PT NOR as Universal gates

NOR

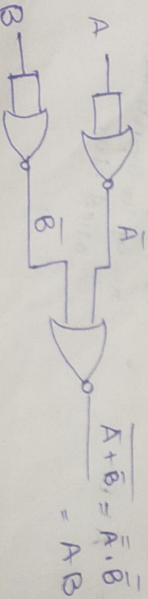
① NOR as a NOT gate



② NOR as a OR gate



③ NOR as a AND gate



24/1/27 Flip Flop (FF)

Flip-Flop is a memory element which can store one bit of information (0 or 1). It has the two stable states & it is also called as bistable multivibrator.

Different types of Flip-Flop

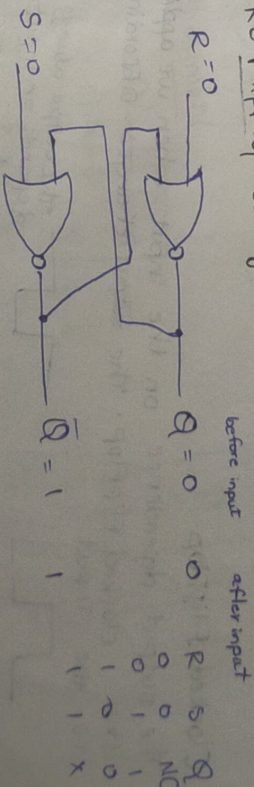
① RS ② D ③ JK ④ T

→ Every Flip-Flop has 2 outputs, they complement to each other (like Q, Q-bar)

→ Set of the Flip-Flop means having output 1. Reset of the Flip-Flop means having output 0.

RS Flip-Flop

RS Flip-Flop using NOR Gate



R=0, S=0 $\Rightarrow$ Q=1, Q-bar=0 No change

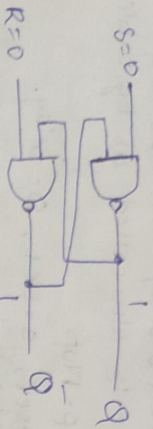
R=0, S=1 $\Rightarrow$ Q=0, Q-bar=1 Set

R=1, S=0 $\Rightarrow$ Q=1, Q-bar=0 Reset

R=1, S=1 $\Rightarrow$ we can't find output (Indeterminate)

29/1/26

RS-FlipFlop using NAND Gate



The outputs are same so it is indetermined

$$e=0, s=0$$

the outputs of after applying inputs

$$S=1$$

$$R=1$$

before $Q=1$ $\bar{Q}=0$ (No change)

$$R=1, S=1, Q=1$$

$$S=1$$

$$R=0$$

Reset $Q=0, \bar{Q}=1$

$$R=0, S=1, Q=0$$

$$S=0$$

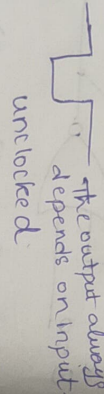
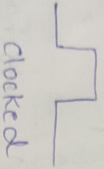
$$R=1$$

Set $Q=1, \bar{Q}=0$

$$R=1, S=0, Q=1$$

Clocked FlipFlop

The output dependence on the input when we applied the clocked FlipFlop. The output changes according to the input.

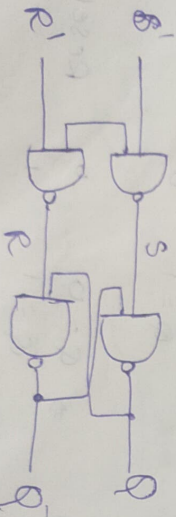


clocked

unclocked

the output always depends on input.

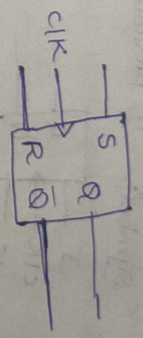
clocked RS FlipFlop using NANDgate



R	S	Q
0	0	NC
0	1	1
1	0	0
1	1	X

30/1/26

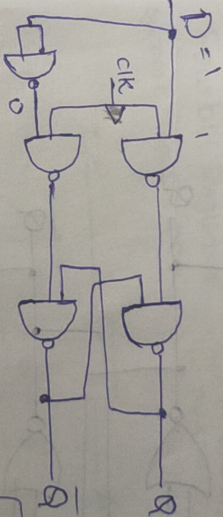
RS-FlipFlop Symbol



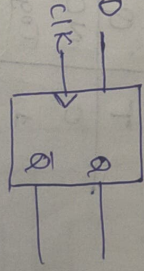
R	S	Q
0	0	NC
0	1	1
1	0	0
1	1	0

D Flip-Flop (most used in registers)

The output follows input $\rightarrow$ if it's just follows input

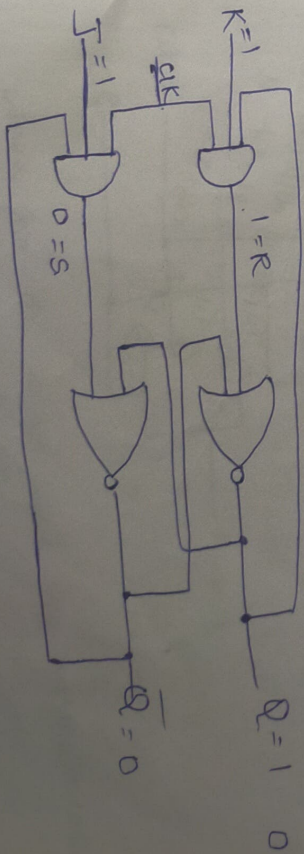


symbol



D	Q
0	0
1	1

JK FlipFlop

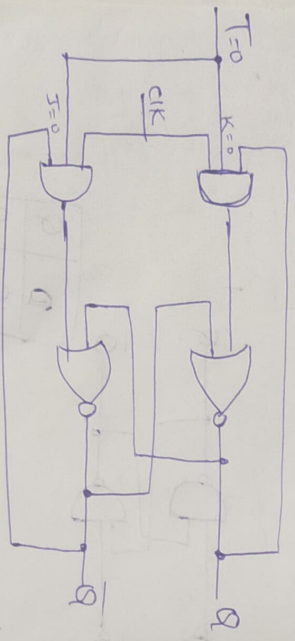


T	K	Q
0	0	NC
1	0	1
0	1	0
1	1	Toggle

Toggle - The present output 0 if previous output 1

Symbol:-

T - Flip-Flop



Symbol:-

T	Q
0	NC
1	Toggle

- ① Types of different registers
- ② Types of shift
- ③ Dec. register

Register

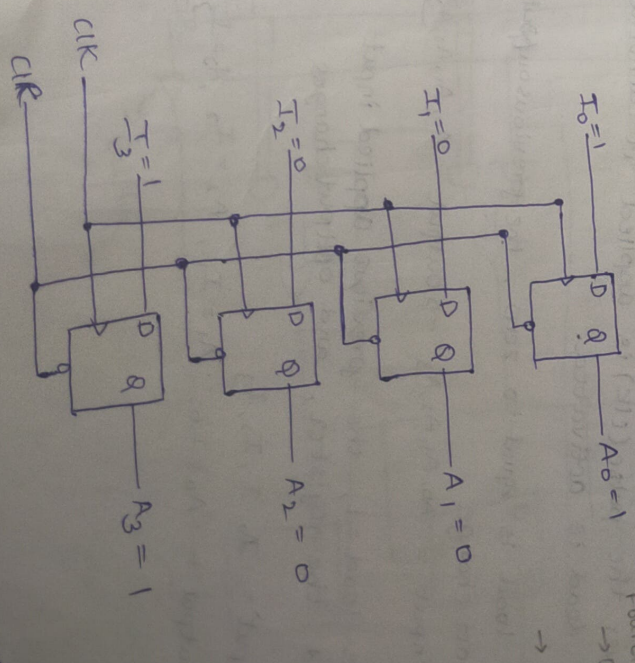
→ A group of bits are called as Binary word.

→ The no. of bits present in Binary word called length of bits

→ A group of FlipFlops connected together to store a Binary word called as Register.

- n bit register contains n FlipFlops.

Four bit Register when clear signal are activated the output become zero

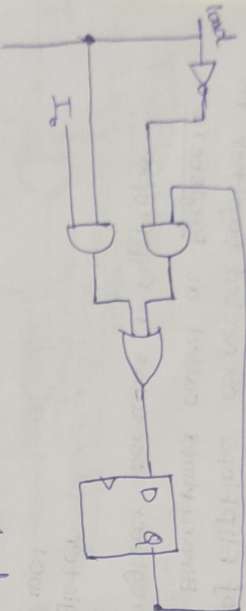


When the clk is activated the automatically output follows input

Four bit Register parallel load

The bits are stored into register simultaneously is called parallel load.

When the load



Whenever the pulse (CLK) is applied it cannot load until load is activated.

→ When load is equal to zero, the previous output does not change

Input = A_0, A_1, A_2, A_3 → Output = A_0, A_1, A_2, A_3

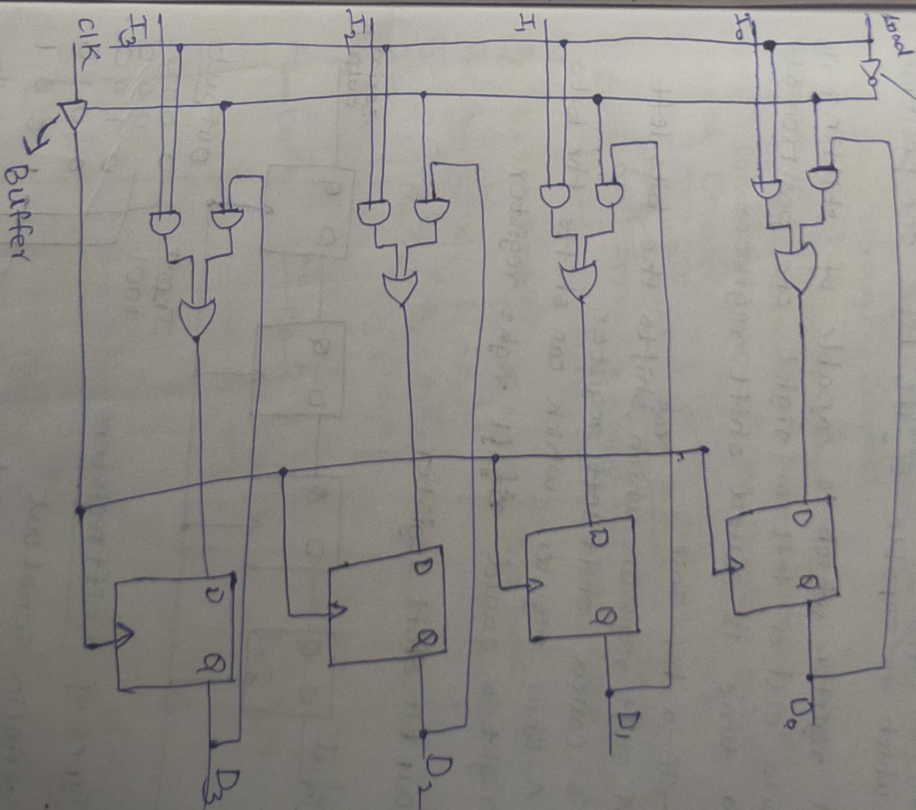
→ When load = 1, the previous output reaches the register, and output changes

Input = A_0, I_1, I_2, I_3

Output = $A_0 = I_0, A_1 = I_1, A_2 = I_2, A_3 = I_3$

31/7/26

Not a test



Shift register

It is a register, it can shift bits ^{one position} which is capable of shifting bits one position.

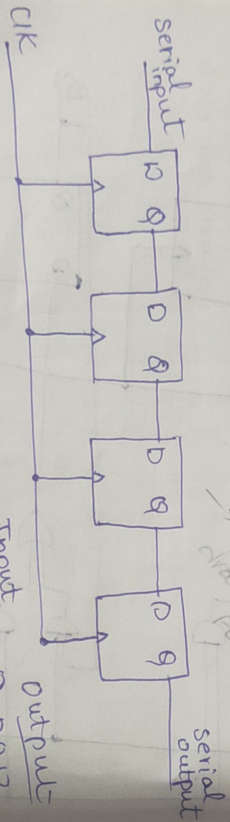
→ A register which is capable of shifting its bits either left or right one position at a time is called shift register.

→ If a bit shift

→ A shift register which shifts the bits left is called shift left register.

→ A shift register which can shift the bits right is called shift right register.

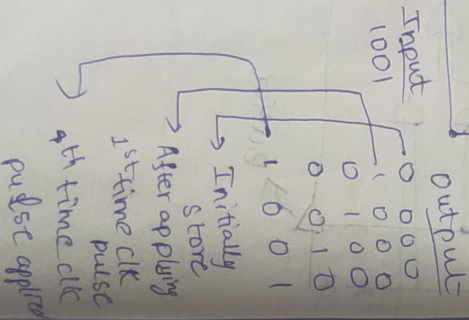
Four bit shift register



This can be drawn for the question four bit register

Types of shift registers

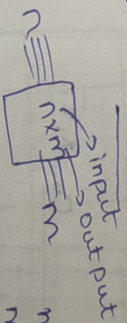
- (1) serial in serial out
- (2) serial in parallel out
- (3) PI
- (4) PISO



Decoder

→ Original data converts to code through n-coder. Again the code converts into data through Decoder.

Decoder is a combinational circuit, which receives the binary information from a coded inputs and converts it into 2^n (max) unique outputs is called Decoder.



n = input size (no. of inputs)
 m = output size (no. of outputs)

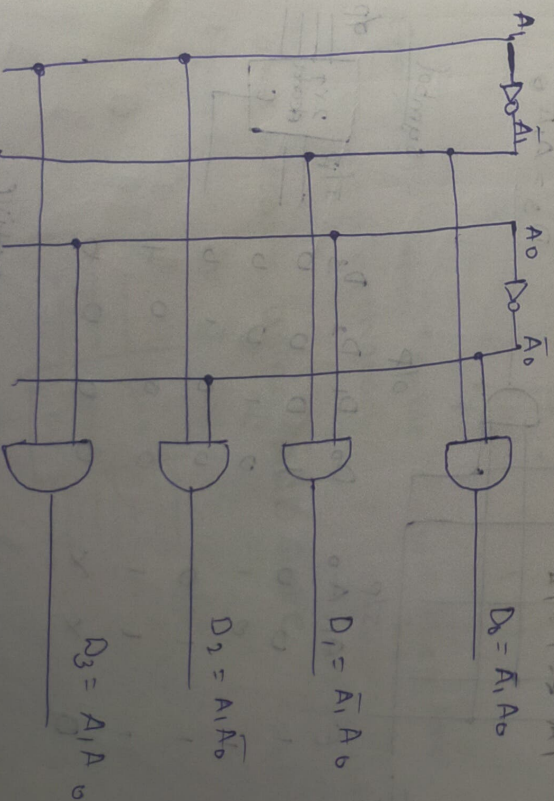
Ex: 2x4 Decoder

Inputs	A_1	A_0
D_3	0	0
D_2	0	1
D_1	1	0
D_0	1	1

$D_0 = \bar{A}_1 \bar{A}_0$
 $D_1 = \bar{A}_1 A_0$
 $D_2 = A_1 \bar{A}_0$
 $D_3 = A_1 A_0$

∴ 2 inputs, so having 4 combinations

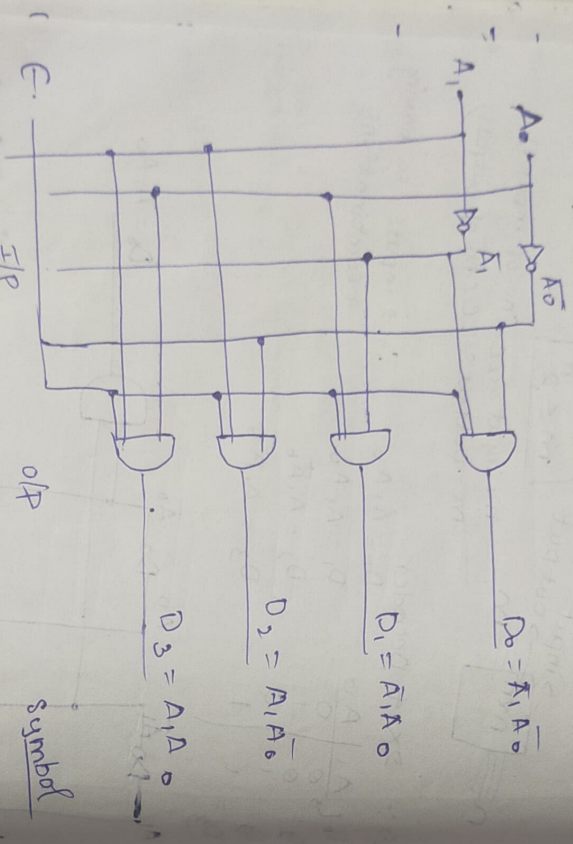
0 - unique code
 $A_1 = D \Rightarrow \bar{A}_1$
 $A_1 = 1 \Rightarrow A_1$



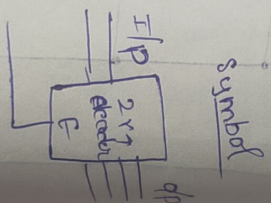
→

I/P		O/P			
A_1	A_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

2 x 4 Decoder with enable input

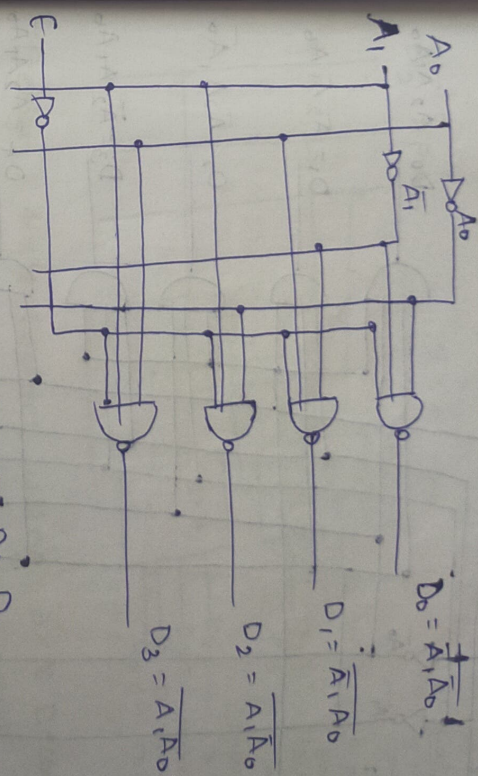


E	A_1	A_0	D_0	D_1	D_2	D_3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0



$E = 1$ = active
 $E = 0$ = inactive

3/8/26
 2x4 Decoder using NAND Gates

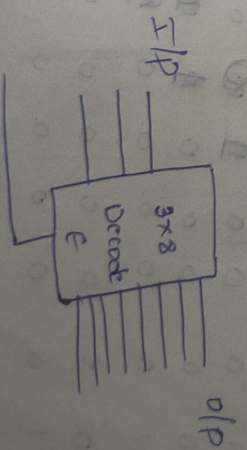


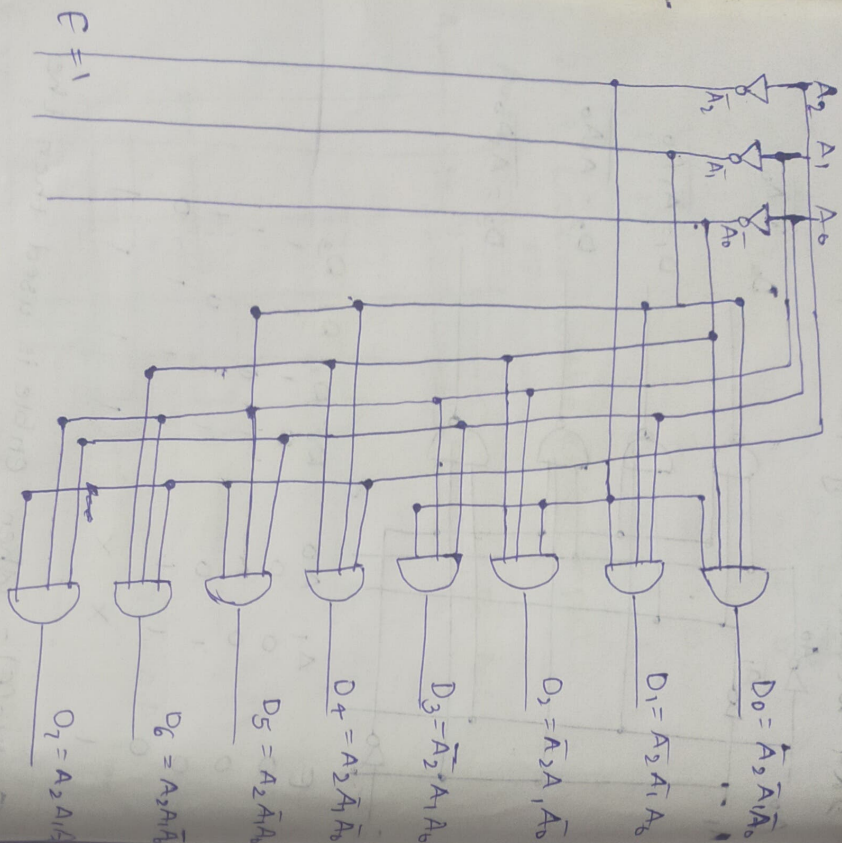
E	A_1	A_0	D_0	D_1	D_2	D_3
0	0	0	1	0	0	0
0	0	1	0	1	0	0
0	1	0	0	0	1	0
0	1	1	0	0	0	1
1	x	x	0	0	0	0

Enable(E) - when enable is used then the circuit is activated

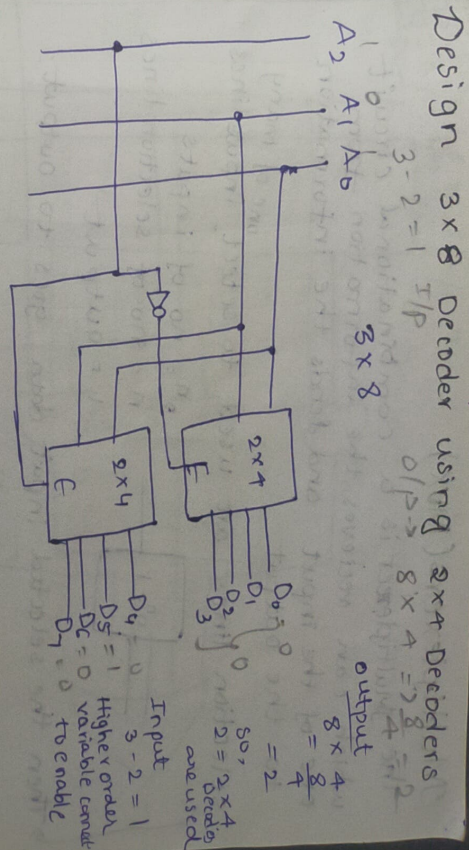
3 x 8 Decoder using AND Gate

Questions like
 Design 2x4
 Decoder using
 AND gate

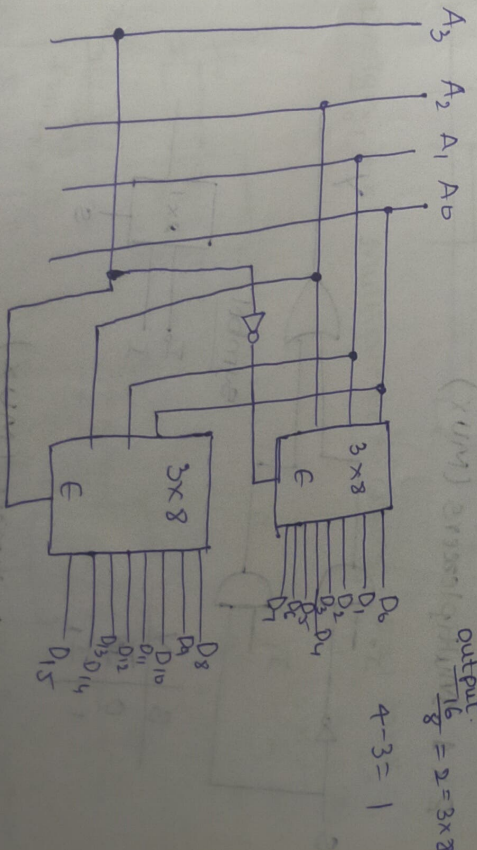




A_2	A_1	A_0	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1



Design 4x16 Decoder using 3x8 Decoders



Multiplexers (MUX)

The Multiplexer is a combinational circuit which can retrieve the information from one of the input and sends the information to the output

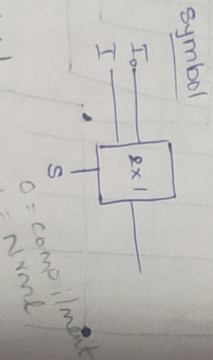
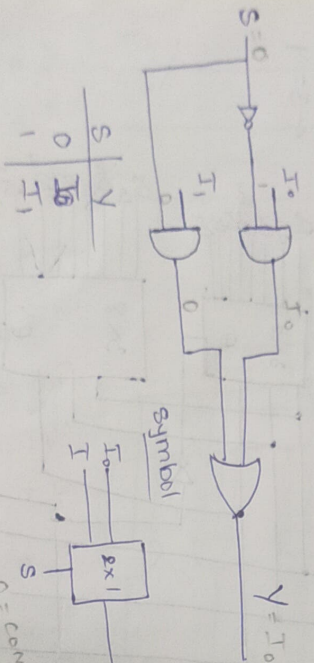
selection lines are used to select one of many inputs



2^n = no. of inputs
 n = no. of selection lines
 1 = output

Then the selected input data goes to output.

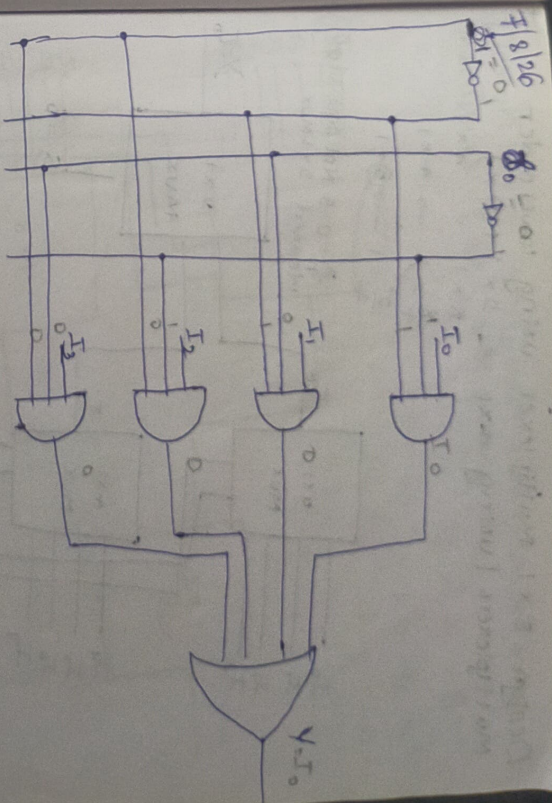
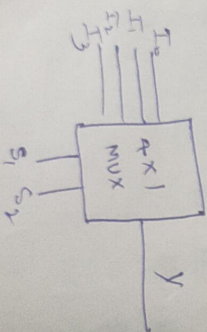
2 x 1 Multiplexers (MUX)



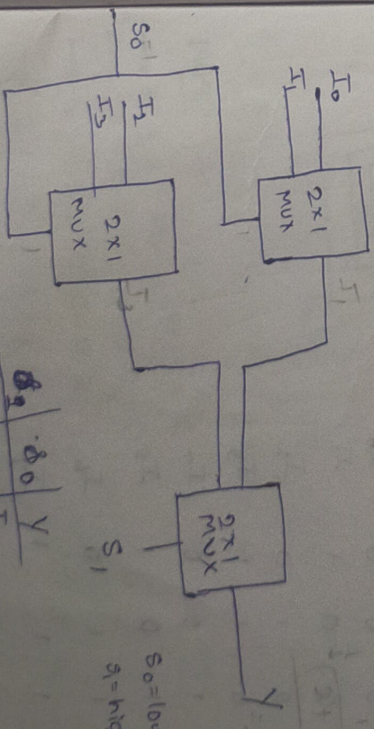
0 = low level
 1 = high level

4 x 1 Multiplexers (MUX)

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



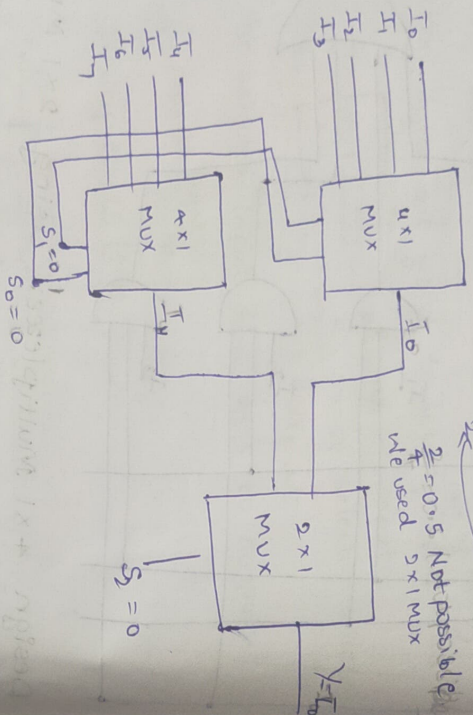
Design 4 x 1 Multiplexer using 2 x 1 MUX



S_0 = low level
 S_1 = high level

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

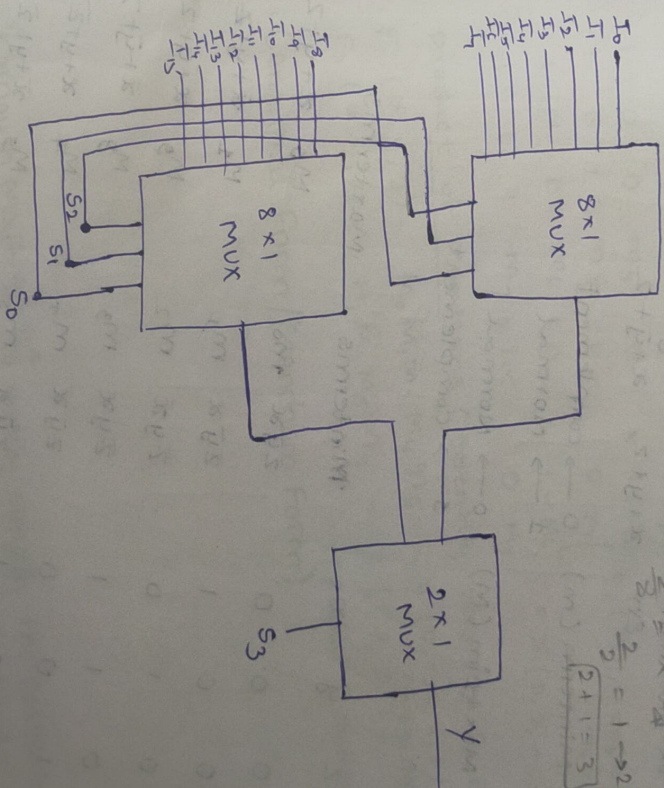
Design 8x1 Multiplexer using lower order Multiplexers (using 4x1 & 2x1 MUX)



$\frac{8}{2} = 4 \rightarrow 4 \times 1$
 $\frac{4}{2} = 2 \rightarrow 2 \times 1$
 $\frac{2}{2} = 1 \rightarrow 1 \times 1$
 $\frac{2}{2} = 0.5$ Not possible
 We used 2x1 MUX

S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

Design 16x1 Multiplexer using lower order Multiplexers (8x1, 4x1 & 2x1)



$\frac{16}{2} = 8 \rightarrow 8 \times 1$
 $\frac{8}{2} = 4 \rightarrow 4 \times 1$
 $\frac{4}{2} = 2 \rightarrow 2 \times 1$
 $\frac{2}{2} = 1 \rightarrow 1 \times 1$
 $\frac{2}{2} = 1 \rightarrow 1 \times 1$

Note:-

- $x + 0 = x$
- $x + 1 = 1$
- $x(y + z) = x \cdot y + x \cdot z$
- $x + yz = (x + y)(x + z)$
- $x + xy = x$
- $x(x + y) = x$
- $x + y = x \cdot y$
- $x \cdot y = x + y$

- Product of Boolean variables, is called minterm or standard product. Represented by 'm'.
- Sum of Boolean variables is called standard sum or maxterm. Represented by 'M'.
- Ex: $x+y+z$, $x+y+z$

∴ Minterm (m)
 0 → complement
 1 → Normal

Maxterm (M)
 0 → Normal
 1 → complement

Z	y	x	Minterms	Maxterms
0	0	0	$\bar{z}\bar{y}\bar{x}$ m ₀	M ₀ $x+y+z$
0	0	1	$\bar{z}\bar{y}x$ m ₁	M ₁ $\bar{x}+y+z$
0	1	0	$\bar{z}y\bar{x}$ m ₂	M ₂ $x+\bar{y}+z$
0	1	1	$\bar{z}yx$ m ₃	M ₃ $x+y+\bar{z}$
1	0	0	$zy\bar{x}$ m ₄	M ₄ $\bar{x}+y+\bar{z}$
1	0	1	zyx m ₅	M ₅ $\bar{x}+y+z$
1	1	0	$z\bar{y}\bar{x}$ m ₆	M ₆ $x+\bar{y}+\bar{z}$
1	1	1	zyx m ₇	M ₇ $\bar{x}+y+\bar{z}$

→ Any Boolean function can be expressed as sum of minterms or the product of maxterms

Like $F(x,y,z) = m_0 + m_3 + m_6 + m_7$
 $= \sum (0, 3, 6, 7)$

$$F(x,y,z) = M_1 M_2 M_4 M_5$$

$$= \prod (1, 2, 4, 5)$$

x	y	z	F ₁	F ₂
0	0	0	m ₀	0
0	0	1	m ₁	0
0	1	0	m ₂	0
0	1	1	m ₃	1
1	0	0	m ₄	0
1	0	1	m ₅	1
1	1	0	m ₆	0
1	1	1	m ₇	1

Product of maxterms (identify zeros in F₁, F₂)
 $F_1(x,y,z) = M_0 M_1 M_2 M_4 M_6 = \prod (0, 1, 2, 4, 6)$
 $F_2(x,y,z) = M_1 M_3 M_5 M_7 = \prod (1, 3, 5, 7)$
 (sum of the product)
 (product of the sum)

Canonical Form / Standard Form

Karnaugh map to easily simplify the Boolean function
 Karnaugh maps are used

→ Two variable map

Two variables → $x, y, z = 2$
 No. of squares = $2^2 = 4 = 2^n$

x \ y	0	1
0	m ₀	m ₁
1	m ₂	m ₃

the term

m₀ = $\bar{x}\bar{y} = 00$
 m₁ = $\bar{x}y = 01$
 m₂ = $x\bar{y} = 10$
 m₃ = $xy = 11$

→ Each square represents two literals

A group of two adjacent squares

→ A two squares are differ by only 1 variables called adjacent squares

10/8/22

- A group of four squares is equal to '1'.
- group of two squares → 1 variable
- single square → 2 variables

For example:-

x	$\bar{x}$	y
1	1	1
0	0	0

$$F(x, y) = \bar{x}\bar{y} + x\bar{y}$$

$$= \bar{x}(\bar{y} + y)$$

$$= \bar{x}$$

Three Variable Karnaugh Map

$$F(x, y, z) \rightarrow 3 \text{ variables} =$$

$$\text{No. of squares } 2^3 = 2^3 = 8$$

x	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	$\bar{y}$	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	$\bar{y}$	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	y	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	$\bar{y}$	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	$\bar{y}$	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	y	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7

$$m_0 = \bar{x}\bar{y}\bar{z} = 000$$

$$m_1 = \bar{x}\bar{y}z$$

$$m_2 = \bar{x}y\bar{z}$$

$$m_3 = \bar{x}yz$$

$$m_4 = x\bar{y}\bar{z}$$

$$m_5 = x\bar{y}z$$

$$m_6 = xy\bar{z}$$

$$m_7 = xyz$$

- single square → 3 variables
- A group of two squares that contains two variables
- A group of 4 squares → 1 variable
- A group of 8 squares → 1 variable

* Only adjacent squares should be grouped that contains one (1).

Four Variable Karnaugh Map

4 variables
 $2^4 = 16 = \text{No. of squares}$

w	x	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}	m_{11}	m_{12}	m_{13}	m_{14}	m_{15}
$\bar{w}$	$\bar{x}$	$\bar{y}$	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}	m_{11}	m_{12}	m_{13}	m_{14}	m_{15}
$\bar{w}$	$\bar{x}$	$\bar{y}$	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}	m_{11}	m_{12}	m_{13}	m_{14}	m_{15}
$\bar{w}$	$\bar{x}$	y	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}	m_{11}	m_{12}	m_{13}	m_{14}	m_{15}
$\bar{w}$	$\bar{x}$	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}	m_{11}	m_{12}	m_{13}	m_{14}	m_{15}
$\bar{w}$	x	$\bar{y}$	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}	m_{11}	m_{12}	m_{13}	m_{14}	m_{15}
$\bar{w}$	x	$\bar{y}$	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}	m_{11}	m_{12}	m_{13}	m_{14}	m_{15}
$\bar{w}$	x	y	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}	m_{11}	m_{12}	m_{13}	m_{14}	m_{15}
$\bar{w}$	x	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}	m_{11}	m_{12}	m_{13}	m_{14}	m_{15}

- * Single square → 4 literals
- * group of 2 squares → 3 literals
- * group of 4 squares → 2 literals
- * group of 8 squares → 1 variable
- * group of 16 squares → '1'

Simplify $F(x, y, z) = \sum (3, 4, 6, 7)$ using K-map

x	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	$\bar{y}$	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	$\bar{y}$	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	y	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	$\bar{y}$	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	$\bar{y}$	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	y	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7

x	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	$\bar{y}$	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	$\bar{y}$	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	y	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	$\bar{y}$	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	$\bar{y}$	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	y	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7

x	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	$\bar{y}$	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	$\bar{y}$	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	y	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
$\bar{x}$	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	$\bar{y}$	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	$\bar{y}$	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	y	$\bar{z}$	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
x	y	z	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7

$$F(x, y, z) = yz + x\bar{z}$$

$$\bar{x}y\bar{z} + x\bar{y}\bar{z}$$

Simplify $f(x, y, z) = \sum (0, 2, 4, 5, 6)$ using K-map

$x \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{x}$	m_0	m_1	m_3	m_2
x	m_4	m_5	m_7	m_6

$x \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{x}$	1			1
x	1	1		1

$$\bar{y}z + x\bar{y}z + y\bar{z}$$

$$f(x, y, z) = \bar{z} + x\bar{y}$$

Simplify $f(x, y, z) = \sum (2, 3, 4, 5)$ using K-map

No. of squares = $2^3 = 8$

$x \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{x}$	m_0	m_1	m_3	m_2
x	m_4	m_5	m_7	m_6

$x \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{x}$				
x	1	1		1

$$f(x, y, z) = x\bar{y} + x\bar{y}z = x\bar{y}$$

Simplify $f(x, y, z) = \sum (3, 4, 6, 7)$ using K-map

$x \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{x}$	m_0	m_1	m_3	m_2
x	m_4	m_5	m_7	m_6

$x \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{x}$				
x		1	1	1

$$f(x, y, z) = yz + x\bar{z}$$

$$x\bar{y}z + x\bar{y}z +$$

14/8/06 Simplify $f(x, y, z) = \sum (1, 4, 5, 6, 7)$ using K-map

$x \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{x}$	m_0	m_1	m_3	m_2
x	m_4	m_5	m_7	m_6

$x \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{x}$		1	1	
x	1	1	1	1

$$f(x, y, z) = \bar{y}z + x\bar{y} + yz$$

$$= \bar{y}z + x(\bar{y} + y)$$

Simplify $f(x, y, z) = \sum (0, 1, 2, 6, 8, 9, 10)$ using K-map

$w \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{w}\bar{x}$	m_0	m_1	m_3	m_2
$\bar{w}x$	m_4	m_5	m_7	m_6
$w\bar{x}$	m_8	m_9	m_{11}	m_{10}

$w \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{w}\bar{x}$			1	1
$\bar{w}x$				
$w\bar{x}$	1	1		
wx				1

$$f(w, x, y, z) = \bar{w}\bar{x}\bar{y} + \bar{w}x\bar{y} + w\bar{x}\bar{z} + w\bar{x}y$$

$$= \bar{x}\bar{y}(\bar{w} + w) + \bar{w}y\bar{z} + w\bar{x}\bar{z}$$

$$= \bar{x}\bar{y} + \bar{w}y\bar{z} + w\bar{x}\bar{z}$$

$$00 + 0010 + 1000$$

$$2 + 8 = 10$$

$$f(w, x, y, z) = \bar{x}\bar{z} + \bar{x}\bar{y}z + \bar{w}y\bar{z}$$

$$00 + 001 + 0010$$

$$1 + 2 = 3$$

$w \backslash yz$	$\bar{y}\bar{z}$	$\bar{y}z$	$y\bar{z}$	yz
$\bar{w}\bar{x}$			1	1
$\bar{w}x$				
$w\bar{x}$	1	1		
wx				1

17/8/26

Simplify $f(w, x, y, z) = \sum (0, 2, 4, 5, 6, 7, 8, 10, 13, 15)$

using K-map

$w \backslash yz$	$y\bar{z}$	$y z$	$\bar{y} \bar{z}$	$\bar{y} z$
$\bar{w} \bar{x}$	m_0	m_1	m_3	m_2
$\bar{w} x$	m_4	m_5	m_7	m_6
$w \bar{x}$	m_{12}	m_{13}	m_{15}	m_{14}
$w x$	m_8	m_9	m_{11}	m_{10}

$w \backslash yz$	$y\bar{z}$	$y z$	$\bar{y} \bar{z}$	$\bar{y} z$
$\bar{w} \bar{x}$	1			
$\bar{w} x$	1	1	1	1
$w \bar{x}$		1		
$w x$	1			1

$$f(w, x, y, z) = \bar{w} \bar{y} \bar{z} + \bar{w} y \bar{z} + \bar{w} x \bar{z} + x \bar{y} \bar{z} + x y \bar{z}$$

$$= \bar{w} \bar{z} (\bar{y} + y) + \bar{w} x \bar{z} + x \bar{z} (y + \bar{y})$$

$$= \bar{w} \bar{z} + \bar{w} x \bar{z} + x \bar{z}$$

Simplify $f(w, x, y, z) = \sum (0, 2, 3, 5, 7, 8, 10, 11, 14, 15)$ using K-map

$w \backslash yz$	$y\bar{z}$	$y z$	$\bar{y} \bar{z}$	$\bar{y} z$
$\bar{w} \bar{x}$	m_0	m_1	m_3	m_2
$\bar{w} x$	m_4	m_5	m_7	m_6
$w \bar{x}$	m_{12}	m_{13}	m_{15}	m_{14}
$w x$	m_8	m_9	m_{11}	m_{10}

$w \backslash yz$	$y\bar{z}$	$y z$	$\bar{y} \bar{z}$	$\bar{y} z$
$\bar{w} \bar{x}$			1	
$\bar{w} x$		1	1	1
$w \bar{x}$				1
$w x$	1			1

$$f(w, x, y, z) = \bar{w} \bar{x} \bar{z} + \bar{w} x \bar{z} + \bar{w} x y + \bar{w} x \bar{z} + \bar{w} x \bar{z} + w \bar{x} \bar{z} + w y$$

$$= \bar{x} \bar{z} (\bar{w} + w) + \bar{w} x y + \bar{w} x \bar{z} + w y$$

$$= w y + \bar{w} x y + \bar{w} x \bar{z} + x \bar{z} \bar{z}$$

Simplify $f(w, x, y, z) = \sum (1, 3, 4, 5, 10, 11, 12, 13, 14, 15)$

using K-map

$w \backslash yz$	$y\bar{z}$	$y z$	$\bar{y} \bar{z}$	$\bar{y} z$
$\bar{w} \bar{x}$	m_0	m_1	m_3	m_2
$\bar{w} x$	m_4	m_5	m_7	m_6
$w \bar{x}$	m_{12}	m_{13}	m_{15}	m_{14}
$w x$	m_8	m_9	m_{11}	m_{10}

$w \backslash yz$	$y\bar{z}$	$y z$	$\bar{y} \bar{z}$	$\bar{y} z$
$\bar{w} \bar{x}$		1		
$\bar{w} x$	1	1		
$w \bar{x}$	1	1		
$w x$			1	1

$$f(w, x, y, z) = \bar{w} \bar{x} \bar{z} + \bar{w} x \bar{z} + \bar{w} x y + w \bar{x} + w x y$$

Simplify $f(w, x, y, z) = \sum (0, 1, 2, 3, 4, 10, 11)$ using K-map

$$= \sum (5, 6, 7, 8, 9, 12, 13, 14, 15)$$

$w \backslash yz$	$y\bar{z}$	$y z$	$\bar{y} \bar{z}$	$\bar{y} z$
$\bar{w} \bar{x}$	m_0	m_1	m_3	m_2
$\bar{w} x$	m_4	m_5	m_7	m_6
$w \bar{x}$	m_{12}	m_{13}	m_{15}	m_{14}
$w x$	m_8	m_9	m_{11}	m_{10}

$w \backslash yz$	$y\bar{z}$	$y z$	$\bar{y} \bar{z}$	$\bar{y} z$
$\bar{w} \bar{x}$		1		
$\bar{w} x$		1	1	1
$w \bar{x}$				1
$w x$	1			1

$$f(w, x, y, z) = (\bar{w} \bar{x} \bar{z}) + \bar{y} \bar{z}$$

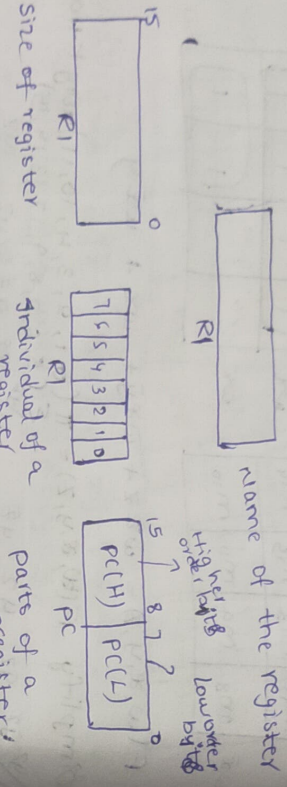
$w \backslash yz$	$y\bar{z}$	$y z$	$\bar{y} \bar{z}$	$\bar{y} z$
$\bar{w} \bar{x}$			1	
$\bar{w} x$	1	1		
$w \bar{x}$				1
$w x$	1			1

$$f(w, x, y, z) = w y + x y + \bar{w} \bar{x} \bar{z}$$

Register Transfer Language (RTL)

* A group of flipflops are connected together to store binary word is called Register.
 * n bit register requires n FF.

Block diagram of Register



RTL

The symbolic representation used to describe the data transfer among the registers is called Register Transfer Language.

Name (symbols)	Purpose	Example
1. Capital letters (numericals)	Name of register	DR, R1
2. parenthesis ()	part of register	PC(L), PC(15:3)
3. $\rightarrow$ (Arrow)	Direction of data transfer	$R_4 \leftarrow R_3$ $R_3 \leftarrow R_1$
4. Comma (,)	To separate the micro operations	$R_2 \leftarrow R_1, R_4 \leftarrow R_3$

Bus

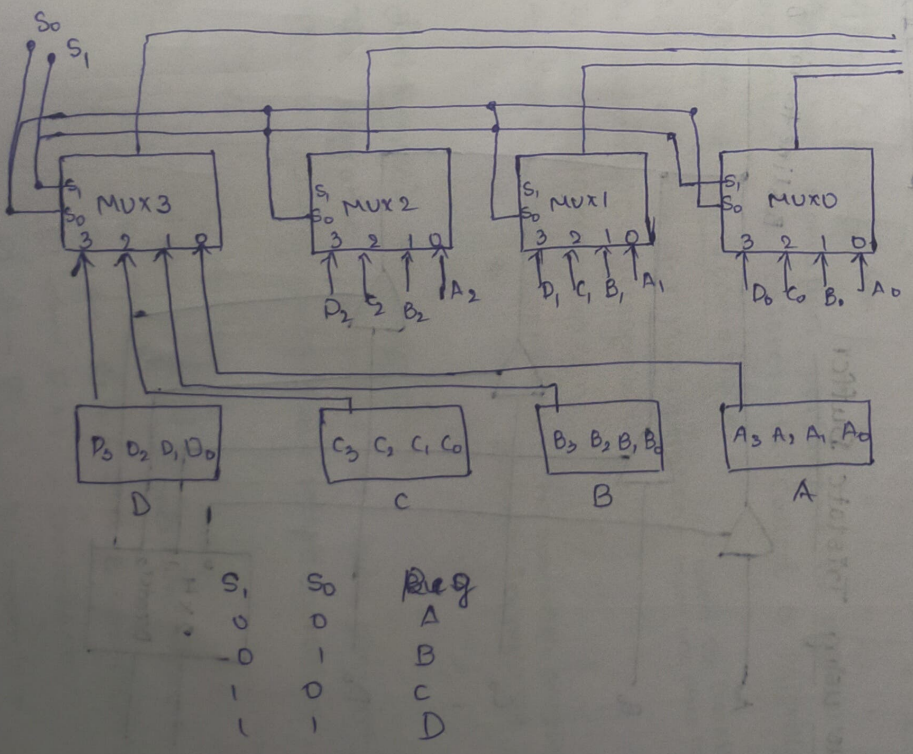
A group of wires are used to transfer the carry the information from one place to another place is called Bus.

We can design bus using decoder & multiplexers.

- 1) Data Bus
- 2) Address Bus
- 3) Control Bus

Types of Bus

Bus using Multiplexers

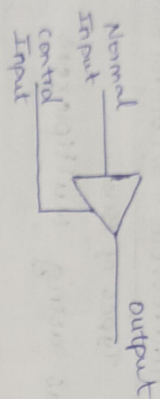


24/8/26

Tristate Buffer

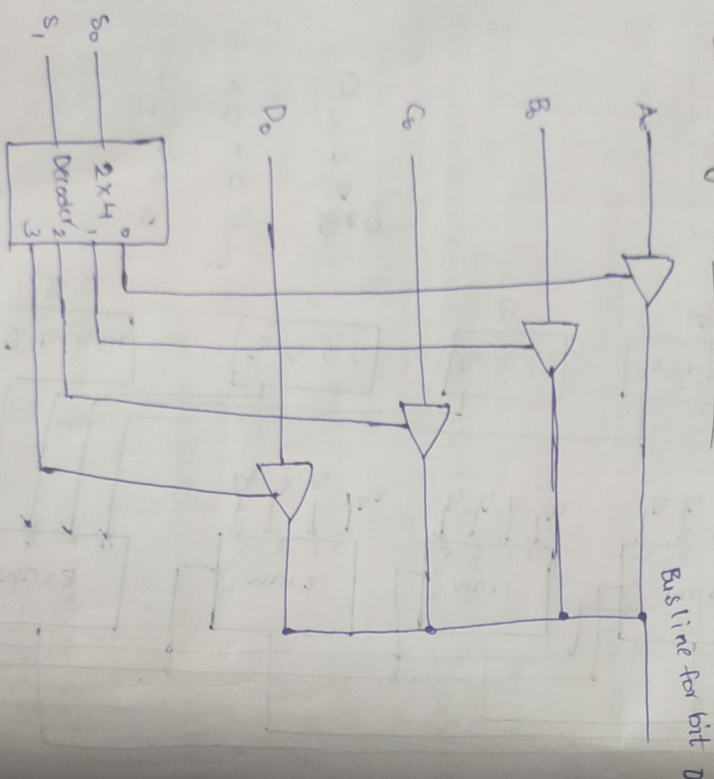
A tristate buffer can have the 3 output states
(low, high, independence)

Symbol



C	Y
0	High
1	output follows input

Bus using Tristate Buffer



Memory Transfer

- * The transfer of data from Memory to external environment or word called as ^{Memory}Read operation.
- * The transfer of data from external world to Memory is called Memory ~~write~~ write

Memory Read $DR \leftarrow M[AR]$

Memory ~~write~~ write $M[AR] \leftarrow R1$

Micro Operation **

The basic operation performed on the data presented in register is called a Microoperation

Types of Microoperations

1. Register Transfer Microoperation - used to transfer data from one place to another
2. Arithmetic Microoperation
3. Logic Microoperation
4. Shift Microoperation - the operation performed on data present in register

Arithmetic Microoperation:

The operation performed on data present in register using Arithmetic Microoperation

Ex:-

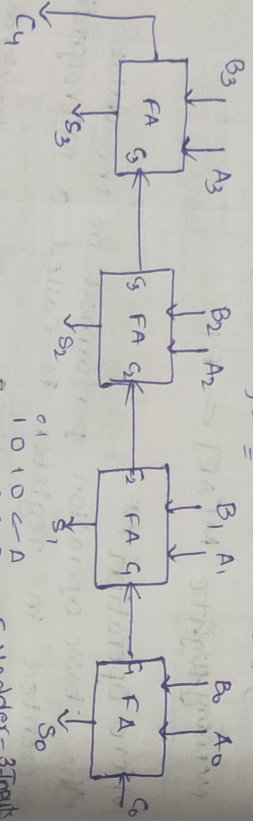
$$\begin{aligned}
 R_3 &\leftarrow R_1 + R_2 \\
 R_3 &\leftarrow R_1 - R_2 \quad \text{--- one's complement} \\
 R_1 &\leftarrow \bar{R}_1 \\
 R_1 &\leftarrow R_1 + 1 \quad \text{--- Two's complement} \\
 R_1 &\leftarrow R_1 + 1 \quad \text{--- Increment} \\
 R_1 &\leftarrow R_1 - 1 \quad \text{--- Decrement}
 \end{aligned}$$

Halfadder - Binary Adder

A digital circuit which can add two binary numbers of any length is called Binary Adder.

∴ n bit Binary adder contains 'n' bit Fulladders

4 Bit Binary adder



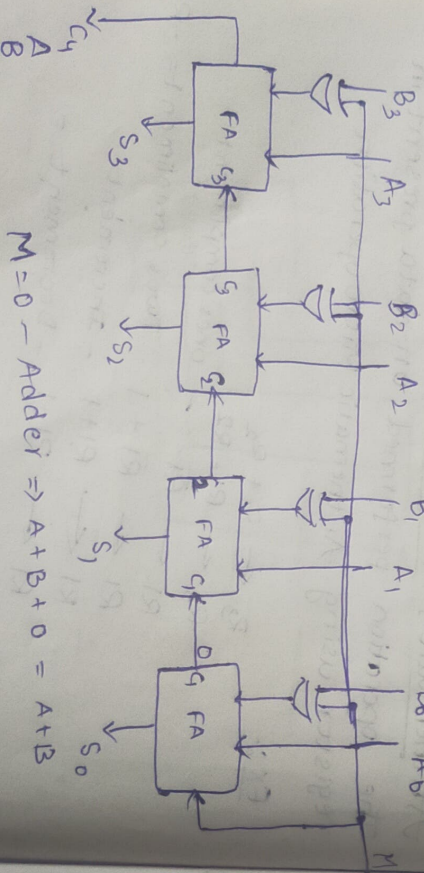
Fulladder - 3 inputs which can add 2 output bits at a time

Adder - Subtractor

It is able to perform the two operations binary addition and binary subtraction but only one operation at a time

Adder - $B \oplus 0 = B$
Subtractor $B \oplus 1 = \bar{B}$

Properties
 $B \oplus 1 = \bar{B}$



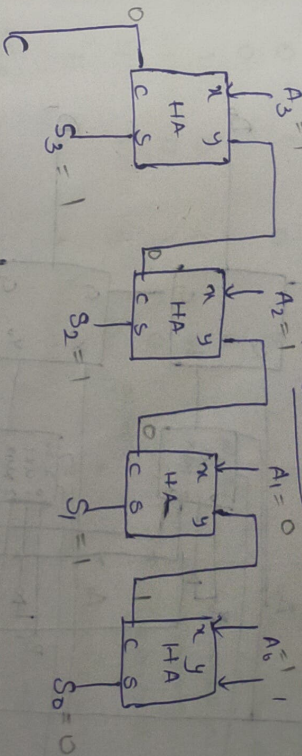
$M=0$ - Adder $\Rightarrow A+B+0 = A+B$

$M=1$ Subtractor $\Rightarrow A+\bar{B}+1 = A-B$

Binary Increment

A digital circuit which increments the value of the given number by 1

4-bit increment



HA - Halfadder - 2 inputs, 2 outputs

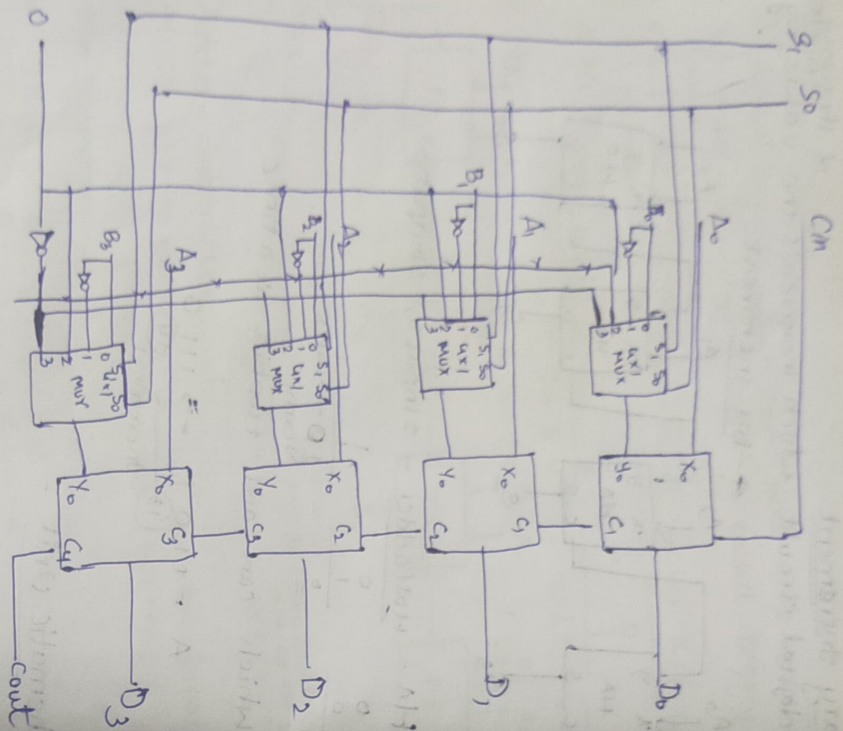
Carry $\leftarrow 1 \Rightarrow \text{Sum} = 0$

→ which can add two bits at a time

Ex- $A = 1101 \rightarrow 1110$

(13) increment (14)

Arithmetic Unit =



Output: $D = A + y + C_{in}$

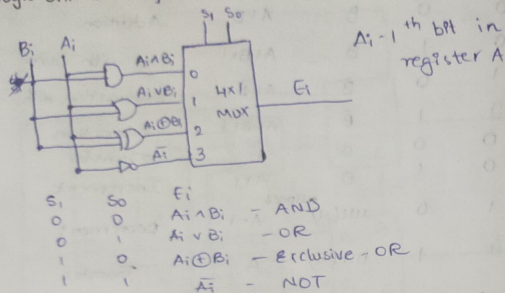
S ₂	S ₁	S ₀	C _{in}	Y	D	
0	0	0	0	B	A+B	Addition
0	0	0	1	B	A+B+1	Add with carry
0	0	1	0	$\bar{B}$	A+ $\bar{B}$	Subtraction with
0	0	1	1	$\bar{B}$	A+ $\bar{B}$ +1	Subtraction with
0	1	0	0	0	A	Transfer
0	1	0	1	0	A+1	Increment
0	1	1	0	0	A-1	Decrement
0	1	1	1	0	A	Transfer

Logic microoperations:

The microoperation is used to perform the logic operations on the data present in a register is called logic microoperations.

- $F \leftarrow 0$ clear
- $F \leftarrow 1$ set
- $F \leftarrow A \vee B$ OR
- $F \leftarrow A \wedge B$ AND
- $F \leftarrow \overline{A \vee B}$ NAND
- $F \leftarrow A \vee \bar{B}$ NOR
- $F \leftarrow A \oplus B$ X-OR
- $F \leftarrow \overline{A \oplus B}$ X-NOR
- $F \leftarrow A$ Transfer
- $F \leftarrow B$ Transfer
- $F \leftarrow \bar{A}$ Transfer
- $F \leftarrow \bar{B}$ Transfer

Logic Unit: 4 operations, one at a time



Applications of Logic operations

Selected set / selective set:

- Set the bits in A if the corresponding bits in B are ones
- perform the OR operation b/w A and B

$$\begin{array}{r} 1100 \leftarrow A \\ 1010 \leftarrow B \\ \hline 1110 \end{array}$$

Selective Complement:

- Complement the bits in A if the corresponding bits in B are ones
- perform the Ex-OR operation b/w A and B

$$\begin{array}{r} 1100 \leftarrow A \\ 1010 \leftarrow B \\ \hline 0110 \end{array}$$

$$\begin{array}{r} 1010 \leftarrow A \\ 1101 \leftarrow B \\ \hline 0111 \end{array}$$

Selective clear:

clear the bits in A if the corresponding bits in B are Ones

$$A \leftarrow A \wedge \bar{B}$$

$$\begin{array}{r} 1100 \leftarrow A \\ 1010 \leftarrow B \\ \hline 0100 \end{array}$$

Mask:

- clear the bits in A if the corresponding bits in B are zeroes
- perform the AND operations b/w A and B

$$\begin{array}{r} 1100 \leftarrow A \\ 1010 \leftarrow B \\ \hline 1000 \end{array}$$

Shift microoperations:

The micro operations are used to shift the bits of a register either left or right are called shift microoperations.

Shift left MO:

shifts the bits left

Shift Right MO:

shifts the bits right

Shift MO are three types:

- logical shift
 - circular
 - Arithmetical
- (*) Logical shift
- (*) Logical shift left (shl)

Each bit shifted left side one bit and the zero will place on the gap.

Ex: R

1	0	1	1
---	---	---	---

 shl R

0	1	0	1
---	---	---	---

(ii) Circular shift

(a) Circular shift left (Cil)

Each bit shifted towards the left side by the one position & left most bit is transferred into the right most position.

Ex:- R

1	0	1	1
---	---	---	---

 cil R

0	1	1	1
---	---	---	---

(b) Circular shift right (Cir):

Each bit is shifted towards the right side by the one position and right most bit is transferred into the left most position.

Ex:- R

1	0	1	1
---	---	---	---

 cir R

1	1	0	1
---	---	---	---

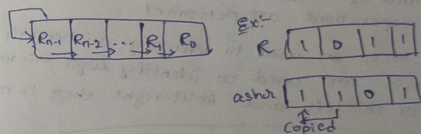
29/8/06

Arithmetic Shift Left

→ Each bit shift towards left side by the one position

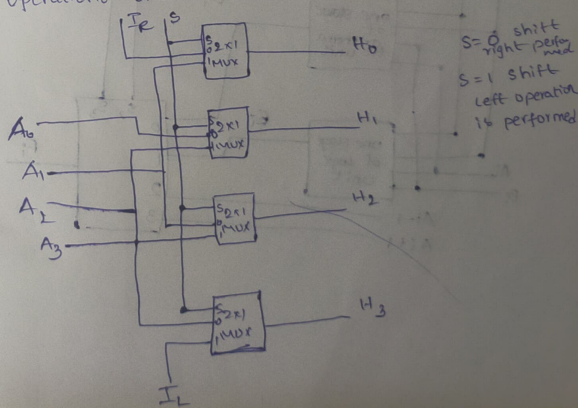
Arithmetic Shift Right (AsR)

→ Each bit shifted towards the right by one position and, sign bit can be copied into itself



Shift unit:

Shift unit is used able to perform different micro operations one at a time.



	H ₀	H ₁	H ₂	H ₃
I _R	A ₀	A ₁	A ₂	A ₃
S=0	I _R	A ₀	A ₁	A ₂
S=1	A ₁	A ₂	A ₃	I _L

★ Arithmetic Logic Shift Unit (14 operations)

→ It is used to perform different Arithmetic Logic, shift ^{micro}operations at a time. That unit is called ALU

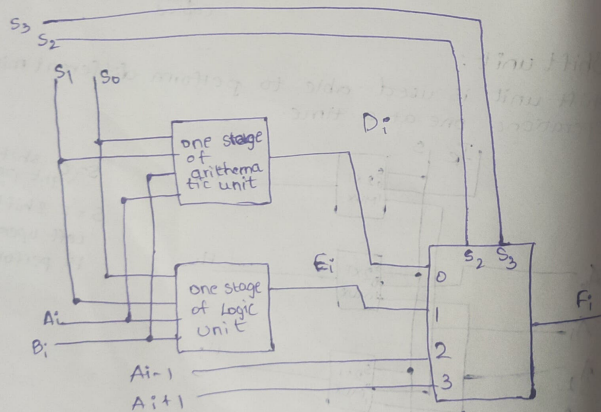
→ Having 4 Selection Lines C_{in}, S₀, S₁, S₂, S₃

→ S₂ and S₃ is used to select which type of microoperations are performed.

→ C_{in}, S₀, S₁ are used to identify arithmetic microoperation

→ S₀, S₁ are used to identify logic microoperation

→ For shift left and shift right there is no identification



S ₃	S ₂	S ₁	S ₀	C _{in}
0	0	0	0	0
0	0	0	0	1
0	0	0	1	0
0	0	0	1	1
0	0	1	0	0
0	0	1	0	1
0	0	1	1	0
0	0	1	1	1
0	1	0	0	x
0	1	0	1	x
0	1	1	0	x
0	1	1	1	x
1	0	x	x	x
1	1	x	x	x

I.

Instruction Code

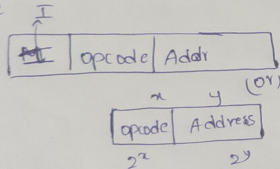
The command given to the computer to do the particular task is called Instruction

A group of the bits that instructs the computer to do the particular operations is called instruction code.

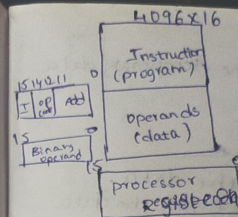
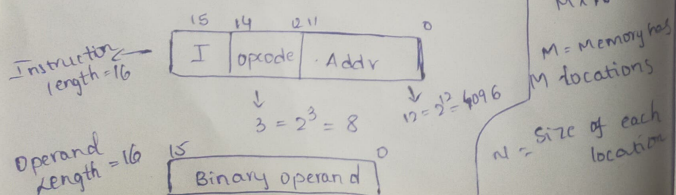
Instruction Code Format:

- Mode - Type of the Address
 - Only one bit is used
 - $M=0 \Rightarrow$ direct
 - $M=1 \Rightarrow$ indirectly addressing
- operation code (Opcode) - A group of bits that specifies operation performed on code

Address - Specifies the address of operators
 n bits perform 2^n operations one operation at a time

Stored program Organization

The ability of a computer to store and execute the instruction



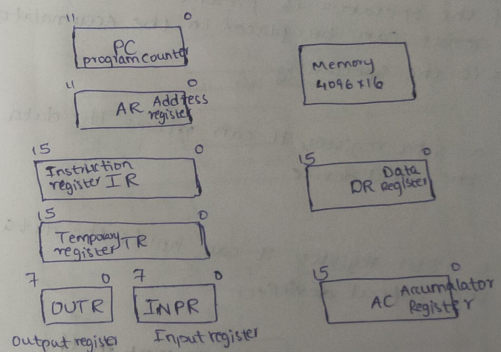
4096x16 = size of the memory

4096 memory locations are present

→ processing happens

* Computer Registers / (Registers in Basic computers)

Basic computer Registers and Memory
 Contains 8 Registers in basic computer

PC

It is a 12 bit register, it contains address of the instruction.

AR

It is a 12 bit register, it contains the address of the memory

IR

It is a 16 bit register, it holds or contains the instruction which reads from the memory.

TR

It is 16 bit register, it holds the temporary data during processing

DR

It is 16 bit register, it holds the data.

* The data can be sent or stored the data called DR.

AC

It is a 16 bit register, it is a processor register.

* One of the operator is present in Accumulator and result can be placed in the Accumulator
↳ this is the speciality.

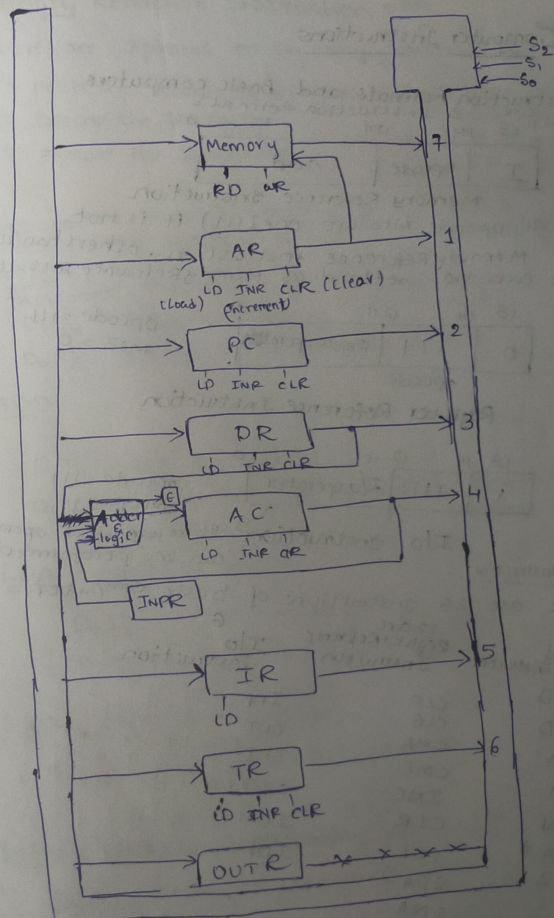
INPR

It is a 8 bit register, it can receives the data from the input device

OUTR

It is a 8 bit register, it can holds the data for the output device.

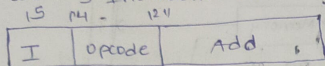
Common Bus (Common Bus connect in the Register and memory in Basic computer)



7/9/26

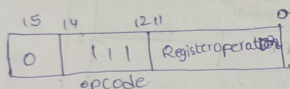
Computer Instructions

Instruction Formats and Basic computers
There are 3 Instruction formats

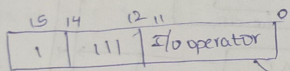


Memory Reference Instruction

All opcode bits are one (111) it is not Memory Reference instruction. other than (111) it can be referred as Memory Reference Instruction.



Register Reference Instruction



I/O Instruction - It refers what operations can be performed

Computer Instruction:

There are 25 instructions of basic computer

7 are Memory Reference Instruction, 12 are Register Reference Instruction, 6 are I/O Instruction

AND	CLA	INP
ADD	CLG	OUT
LDA	CMA	SKI (Input flag = 0 skip the next instruction)
STA	CME	SKP (Output flag = zero 11)
BUN	INC	ION (to enable)
BSA	CIR	IDF (to disable)
ISZ	CIL	
	SPA	
	SNA	
	SZA	
	SZE	
	H&T	(it doesn't run any instruction)

Increment the value if incremented value is equal to zero then skip next one. If not equal to zero then go to next one.

Memory Reference Instruction

There are 3 phases to run any instruction

1. Fetch the instruction - getting instruction
2. Decode the instruction
3. Execute the instruction

These are the microoperation to execute the instruction

AND: These are the microoperations of AND

D₀T₄: DR ← M[AR]
D₀T₅: AC ← AC ∧ DR, SC ← 0

ADD

D₁T₄: DR ← M[AR]
D₁T₅: AC ← AC + DR, E ← Cout, SC ← 0
(enable) (sequence count)

LDA - Load Accumulator

D₂T₄: DR ← M[AR]
D₂T₅: AC ← DR, SC ← 0

STA - Store Accumulator

D₃T₄: M[AR] ← AC, SC ← 0

BUN - Branch Unconditionally

D₄T₄: PC ← AR, SC ← 0

BSA - Branch save and return Address. Return address in memory & increment the address it is stored in AR again it was stored in PC

D₅T₄: M[AR] ← PC, AR ← AR + 1

D₅T₅: PC ← AR, SC ← 0

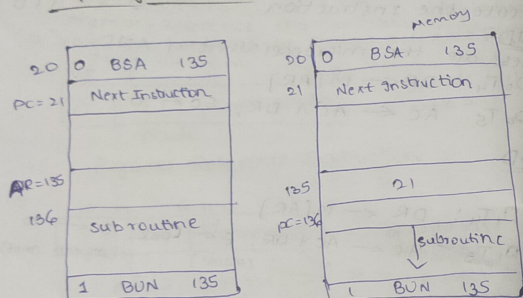
ISZ : Increment skip Zero

D₆ T₄ : $DR \leftarrow m[AR]$

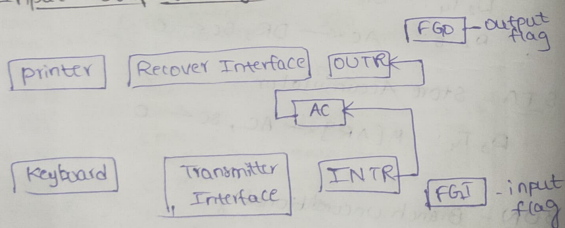
D₆ T₅ : $DR \leftarrow DR + 1$

D₆ T₆ : $M[AR] \leftarrow DR$, if $(DR=0)$ then $(PC \leftarrow PC+1)$, $SC \leftarrow 0$

Example of the BSA:



Input - output Configuration



$FGI = 0$, then Register-Empty

$FGI = 1$, then Register-Non-empty

$FGO = 1$, Register Empty

$FGO = 0$, Register non-empty

OUTR: Communicate '11el to AC & serial to printer

INTR: " " & " " Keyboard

Process:

- 1) clear the I/p flag
- 2) select any key in Keyboard
- 3) then the code of that key is travelled to Input Register through Transmitter interface.
- 4) INTR is not empty so FGI becomes 1.
- 5) Whenever it becomes 1, AC (cpo) receives data from INPR '11el
- 6) then FGI becomes zero
- 7) If $FGO = 1$, the output BUTR receives data from AC and FGO becomes zero
- 8) Add it travels to printer through Recover interface.

Instructions cycle:

The time taken to perform the no. of operations in a particular sequence to run a instructions is called instruction cycle

phases of instruction cycle

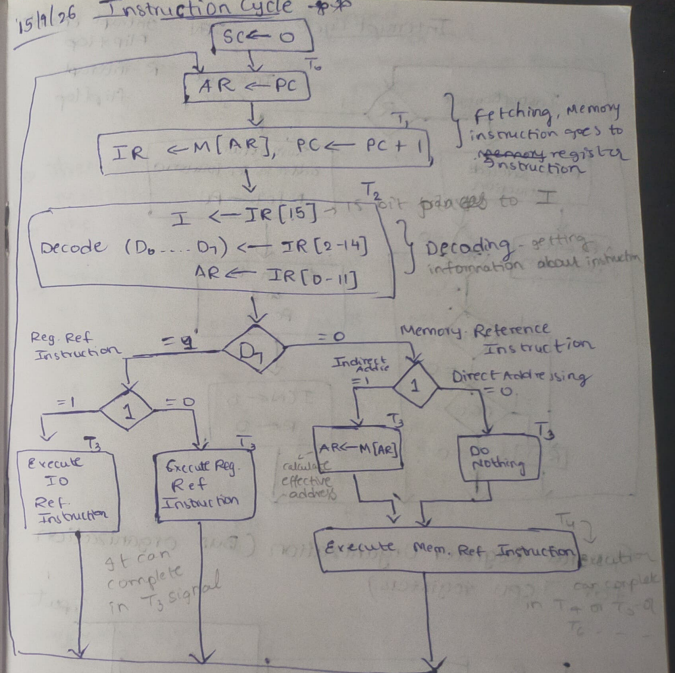
(i) Fetch (ii) Decode (iii) calculate EA

(iv) Execution-

- ① process of getting instruction from memory and placed in memory phase

- ② After Fetch- Decode (To know more details about Instruction)
- ③ After Decode - Instructions some (which operation to perform what type of operation it is - - -)
- ④ Execute the instruction.

15/9/26 Instruction Cycle

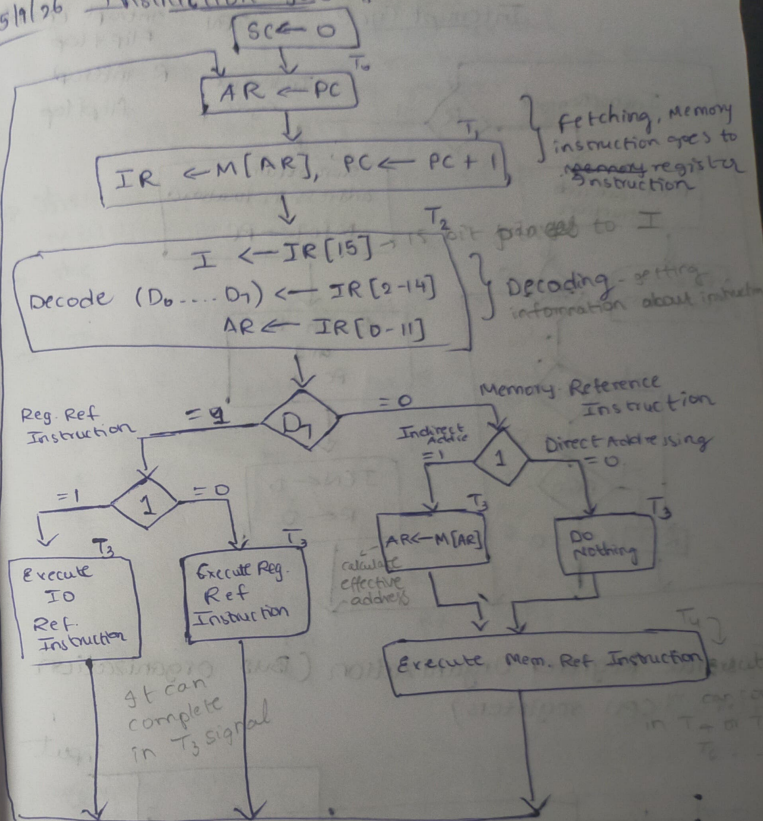


Program Interrupt

Program Interrupt

- ② After Fetch- Decode (To know more details about Instruction)
- ③ After Decode - Instructions some (which operation to perform what type of operation it is ---)
- ④ Execute the instruction.

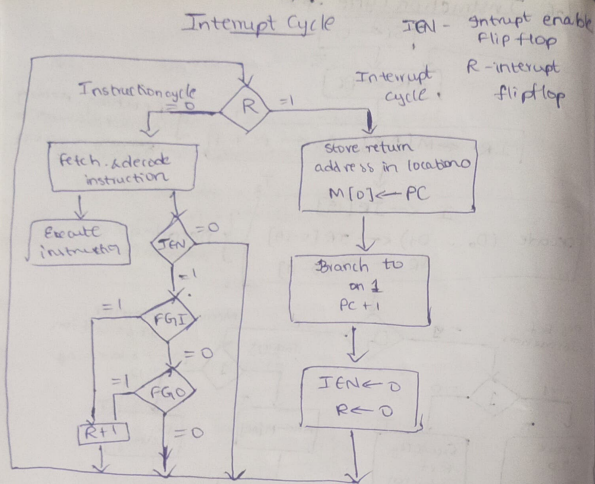
15/9/96 Instruction Cycle



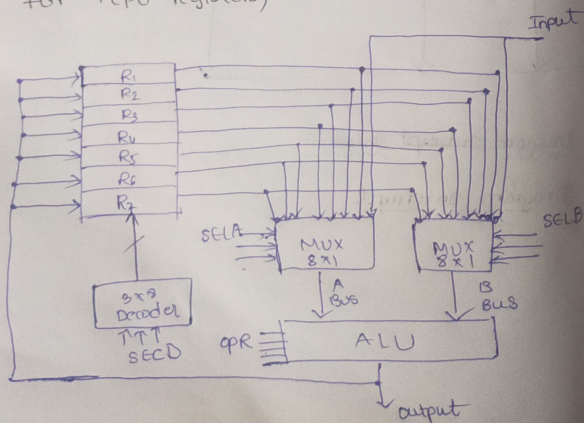
Program Interrupt

Program interrupt

Interrupt Cycle



General Register Organization (Bus organization for 7 CPU registers)



SELA - selecting one operand
SELB - selecting another operand
SELD - where the result stored goes to, selects Destination register
OPR - selecting what operation to be done.

- The o/p of Registers connected to 3p of Muxers
- the o/p of Multiplexers connected to 3p of ALU
- The o/p of ALU connected to 3p of registers

	SELA	SELB	SELD
0 0 0	Input	Input	None
0 0 1	R ₁	R ₁	R ₁
0 1 0	R ₂	R ₂	R ₂
0 1 1	R ₃	R ₃	R ₃
1 0 0	R ₄	R ₄	R ₄
1 0 1	R ₅	R ₅	R ₅
1 1 0	R ₆	R ₆	R ₆
1 1 1	R ₇	R ₇	R ₇

Control Word:

3	3	3	5bits
SELA	SELB	SELD	OPR

0 0 0 0 Transfer operation
0 0 0 1 Increment operation
0 0 1 0 Addition operation
0 0 1 1 Subtraction operation

Ex- ① $R_6 \leftarrow R_1 + R_2$

Control word-

SELA	SELB	SELD	OPR
001	010	110	00010

Control word for following operation

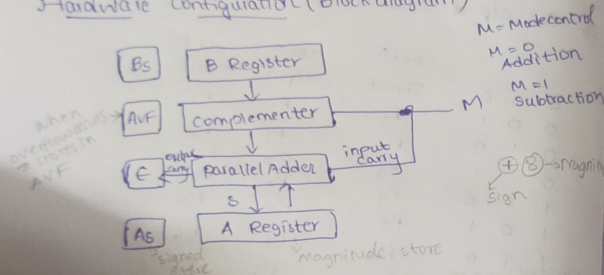
Input + R7 →

R2 ← Input + R7

000 | 111 | 010 | 00010

Addition and subtraction with signed magnitude data

Hardware Configuration (Block diagram)



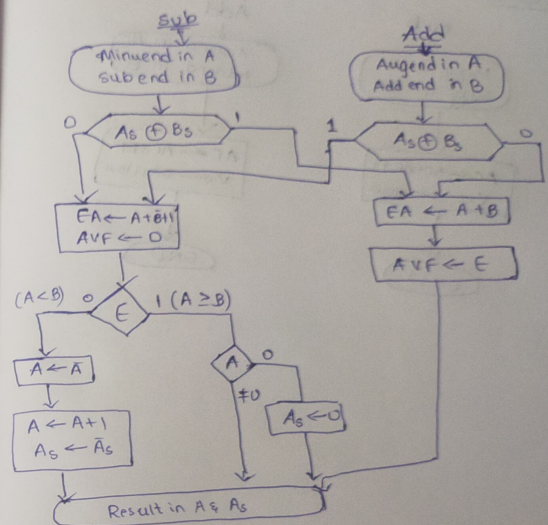
Algorithm

Addition (subtraction)

- If signs are identical (different), add the magnitudes and assign the sign of the one number to the result.
- If signs are different (identical), subtract the smaller number from the larger number and assign the sign of the larger number to the result.

- Only in Addition overflow occurs
- $M=0$, the complement work as Buffer
- $M=1$, the complement work as complement

Flowchart



Addition and subtraction with 2's complement representation of the data:

Hardware Configuration

